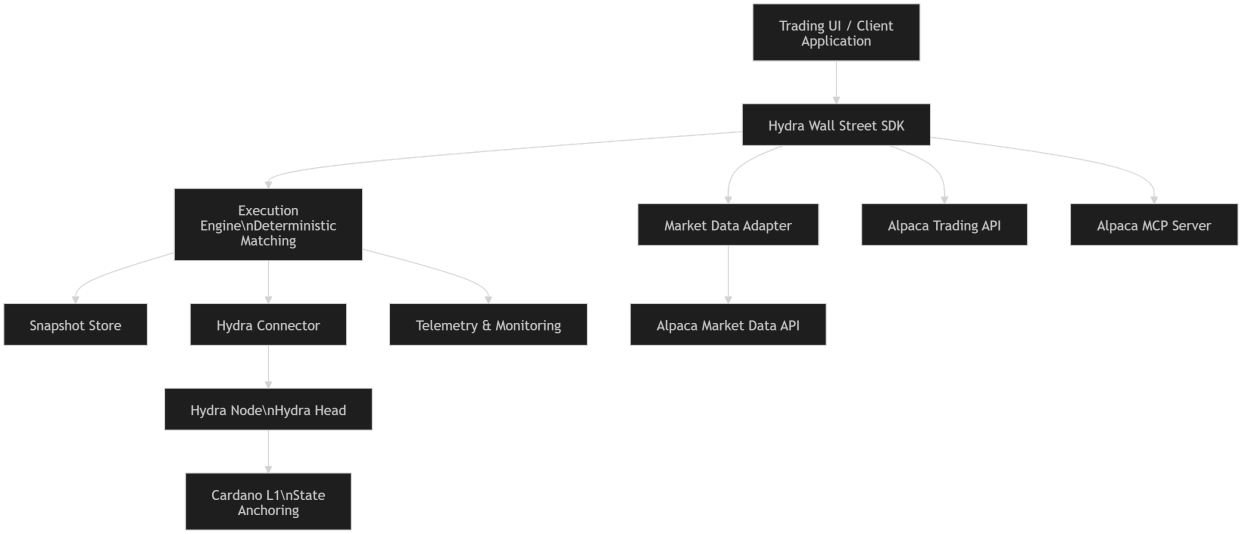
# Hydra Wall Street Library - Architecture Documentation

## 1. System Overview




The **Hydra Wall Street Library** is a deterministic financial execution simulation framework built on **Hydra Heads**, enabling developers to simulate trading environments with high throughput and deterministic replay.

The system provides:

- financial market simulation tools
- deterministic order execution
- reproducible replay environments
- SDKs for developer integration
- telemetry for performance benchmarking
- integration with real-world financial trading APIs

Hydra Heads act as the **low-latency execution environment**, while Cardano L1 provides **state anchoring and settlement guarantees**.

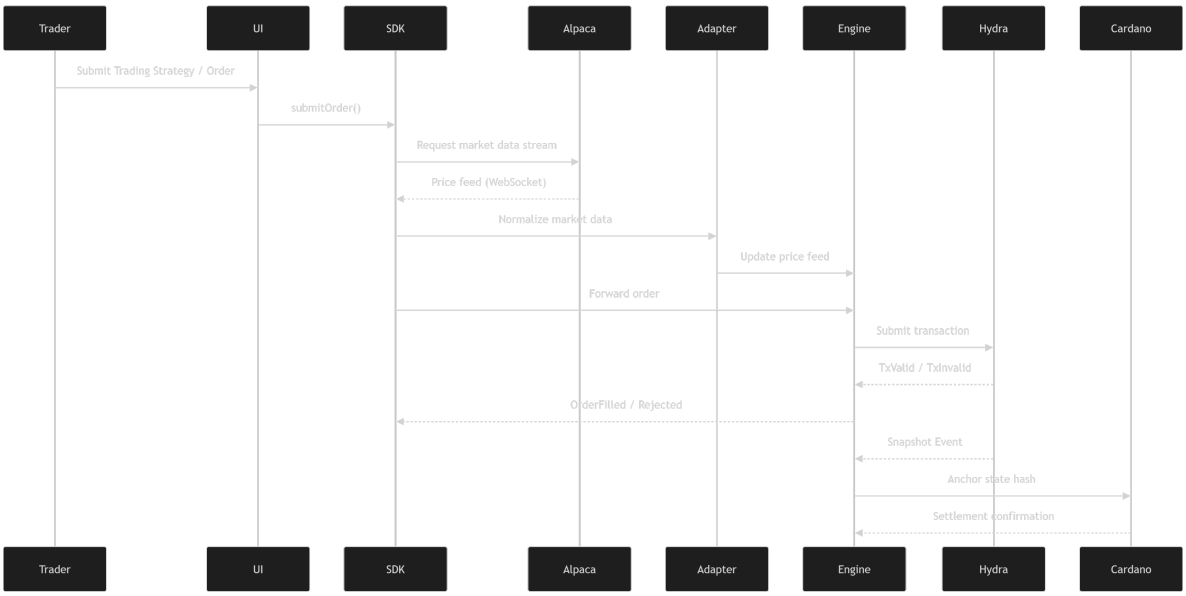
To simulate realistic financial environments, the system integrates with **external market infrastructure providers**, including:

- **Alpaca Trading API**
- **Alpaca Market Data API**
- **Alpaca MCP Server**

These integrations allow the Hydra Wall Street Library to ingest **live market data and simulated brokerage execution flows** from a widely used algorithmic trading platform.

---

# 2. Execution Workflow (End-to-End)




Normal operation follows a deterministic trading lifecycle:

```
Connect → Spawn / Join Head → Snapshot Sync
→ Market Data Feed Integration → Order Submission
```

```
→ Matching Engine Execution → Snapshot Anchoring
→ Head Close → Settlement
```

---

## 2.1 Connect

The SDK establishes a WebSocket connection with the Hydra node.

Steps:

1. Client initiates connection
2. SDK establishes WS connection
3. Hydra node begins streaming events
4. Client restores last snapshot state

Additionally, the SDK may establish connections with **external market data providers**, such as Alpaca.

```
Client
↓
Hydra SDK
↓
Hydra WebSocket
↓
Hydra Node
```

Optional external connection:

```
Hydra SDK
↓
Alpaca Market Data WebSocket
```

---

## 2.2 Spawn or Join Head

The system either creates or joins a Hydra Head.

Example flow:

```
Client → SDK → Hydra HTTP API → Hydra Node
```

Endpoints:

```
POST /head/spawn
POST /head/join
GET /head/status
```

When the head opens, clients subscribe to lifecycle events.

---

## 2.3 Snapshot Synchronization

After joining the head:

1. Hydra sends **Snapshot event**
2. SDK initializes local execution state
3. Execution engine reconstructs order book
4. Event pipeline begins processing transactions

Snapshots act as the **authoritative state baseline**.

External market data streams may also initialize simulation state.

Example:

```
Alpaca Market Data
↓
Market Data Adapter
↓
Hydra Execution Engine
```

---

## 2.4 Order Submission

Orders are submitted through SDK APIs.

Example order lifecycle:

```
Client
   ↓
SDK
   ↓
Execution Engine
   ↓
Hydra WebSocket
   ↓
TxValid / TxInvalid
```

Order types supported:

- Limit
- IOC
- Cancel
- Partial fill

In simulation environments, **orders may be derived from Alpaca trading signals or strategy outputs**, allowing developers to test Wall Street–style strategies within Hydra Heads.

---

## 2.5 Matching Engine Execution

The deterministic matching engine performs:

- price-time priority matching
- order book updates
- trade generation
- position tracking

Execution events emitted:

```
OrderAccepted
OrderRejected
OrderFilled
```

```
OrderPartiallyFilled
OrderCancelled
```

Market prices may be updated through **Alpaca Market Data streaming feeds**.

```
Alpaca Market Data API
↓
Market Data Adapter
↓
Hydra Matching Engine
```

This allows Hydra to simulate **realistic financial market conditions**.

---

## 2.6 Snapshot Anchoring

Periodic snapshots are emitted by Hydra.

Process:

```
Hydra Snapshot
↓
Execution Engine State Hash
↓
Snapshot Store
↓
Optional Cardano L1 Anchor
```

This ensures deterministic replay.

Snapshots can also include **market data checkpoints derived from Alpaca feeds**.

---

## 2.7 Head Close and Settlement

Closing the Hydra Head triggers settlement.

Steps:

```
Client initiates close
↓
Hydra node finalizes state
↓
Final snapshot generated
↓
State anchored on Cardano L1
```

The session state is then cleared.

---

# 3. Failure & Recovery Paths

The architecture includes deterministic recovery mechanisms.

---

## Network Disruption

Scenario:

```
Network Drop
↓
WS disconnect detected
↓
Auto-reconnect attempt
↓
Request latest snapshot
↓
Replay missed events
```

External API disruptions are also handled.

Example:

```
Alpaca API interruption
↓
Fallback to cached market data
```

```
↓
Reconnect to market data stream
```

This ensures deterministic recovery.

---

## Snapshot Loss

If events are received without a snapshot baseline:

```
Reject event processing
↓
Request snapshot resync
↓
Rebuild execution state
```

---

## Event Ordering Issues

To ensure deterministic behavior:

- events buffered
- duplicates removed
- strict sequencing enforced

---

# 4. API & Integration Points

The system exposes multiple interfaces.

---

# 4.1 Hydra Node Interfaces

### WebSocket (Real-Time Events)

Events streamed:

```
Snapshot
TxValid
TxInvalid
HeadOpened
HeadClosed
OrderFilled
```

These events drive the execution engine state.

---

### HTTP Control APIs

Control-plane actions are performed through HTTP endpoints.

```
POST /hydra/head/spawn
POST /hydra/head/join
POST /hydra/head/close
GET  /hydra/head/status
```

These APIs manage Hydra lifecycle operations.

---

# 4.2 SDK Public Interface

SDK functions include:

```
connect()
spawnHead()
joinHead()
submitOrder()
cancelOrder()
subscribeEvents()
getOrderBook()
subscribeMarketFeed()
```

Supported SDKs:

- TypeScript
- Python

These SDKs abstract Hydra complexity for developers.

---

# 4.3 Alpaca Integration

The Hydra Wall Street Library integrates with **Alpaca's developer platform** to simulate realistic trading environments.

## Alpaca MCP Server

The **Alpaca MCP Server** provides a standardized interface for connecting trading strategies, AI agents, and automation tools to Alpaca's trading infrastructure.

Hydra SDK can interface with MCP Server to:

- ingest trading signals
- simulate brokerage order flow
- replay algorithmic trading strategies

Example flow:

```
Trading Strategy
↓
Alpaca MCP Server
↓
Hydra Wall Street SDK
↓
Hydra Execution Engine
↓
Hydra Head
```

This allows developers to test **Wall Street–style algorithmic strategies on Hydra infrastructure**.

---

## Alpaca Trading API

Alpaca's Trading API allows developers to interact with brokerage infrastructure.

Hydra integration may include:

```
GET /v2/account
GET /v2/orders
POST /v2/orders
DELETE /v2/orders/{id}
```

Use cases:

- strategy signal generation
- order simulation
- brokerage workflow testing

Orders from Alpaca strategies can be **mirrored inside Hydra execution environments**.

---

## Alpaca Market Data API

Alpaca Market Data API provides real-time financial data streams.

Supported feeds include:

- equities price feeds
- trade events
- order book updates

Example integration:

```
Alpaca Market Data WebSocket
↓
Market Data Adapter
↓
Hydra Matching Engine
```

This enables Hydra to simulate **real-world market price movement**.

---

## 4.4 Market Data Integration

Market data adapters provide normalized feeds.

Sources supported:

- deterministic synthetic generators
- historical replay datasets
- Alpaca Market Data API
- external financial APIs

Adapters convert feeds into a standard schema.

---

# 5. Security & Scalability

The architecture includes multiple security and scalability safeguards.

---

## 5.1 Secure Transport

All connections use:

```
TLS WebSocket
HTTPS APIs
```

This includes:

- Hydra node connections
- Alpaca API connections

Certificate validation prevents man-in-the-middle attacks.

---

## 5.2 Replay Protection

Replay attacks are prevented through:

- sequence numbers
- snapshot verification

- deterministic event ordering

## 5.3 Message Validation

All inbound events undergo:

- schema validation
- transaction integrity checks
- order validation

Invalid events are rejected.

## 5.4 Scalability Controls

The system ensures Hydra throughput is preserved.

Controls include:

- buffered event pipeline
- non-blocking WebSocket processing
- rate-limited SDK calls
- external API throttling management

The execution layer is designed **not to bottleneck Hydra throughput**.

# 6. Risk Evaluation & Mitigation

| Risk | Impact | Mitigation |
|---|---|---|
| Sync loss | State divergence | Snapshot-driven recovery |
| Event ordering errors | Inconsistent execution | Sequenced event processing |
| Market data gaps | Invalid simulation | Deterministic synthetic feeds |
| Alpaca API outage | Data unavailability | Cached feed fallback |
| Replay inconsistencies | Incorrect backtests | Hash-based verification |

These strategies ensure reliable financial simulation.

# 7. System Architecture Blueprint

## Module Layout

The architecture is composed of the following components:

```
Application UI
↓
SDK Layer
↓
Execution Engine
↓
Market Data Adapter
↓
Hydra Connector
↓
Hydra Node
↓
Cardano L1
```

External services:

```
Alpaca MCP Server
Alpaca Market Data API
Alpaca Trading API
```

---

# 8. Component Responsibilities

## Application UI

Provides visualization tools:

- order book depth
- trade tape
- P&L tracking

- latency monitoring

---

## SDK Layer

Developer-facing API layer.

Responsibilities:

- connection management
- event subscriptions
- order submission
- lifecycle orchestration
- external API integration

---

## Execution Engine

Implements deterministic financial simulation.

Functions:

- order matching
- state updates
- event emission

---

## Market Data Adapter

Normalizes external data sources.

Responsibilities:

- ingest Alpaca feeds
- normalize price feeds
- feed execution engine

---

## Hydra Connector

Manages communication with Hydra nodes.

Responsibilities:

- WS event ingestion
- HTTP control commands
- reconnect handling

---

### Snapshot Store

Stores execution checkpoints.

Responsibilities:

- snapshot persistence
- replay initialization
- divergence detection

---

### Anchoring Service

Computes state hashes and anchors results on Cardano.

---

# 9. Client State Machine

Execution state transitions follow the Hydra lifecycle.

```
Disconnected
   ↓
Connected
   ↓
HeadOpen
   ↓
SnapshotSynced
   ↓
ActiveExecution
   ↓
Closing
   ↓
```

```
Fanout
   ↓
Settled
```

If network failure occurs:

```
ActiveExecution → Disconnected → Reconnect → SnapshotReplay →
ActiveExecution
```

---

# 10. Message Flow (SDK ↔ Hydra)

Transaction flow:

```
Client
↓
SDK
↓
Execution Engine
↓
Market Data Adapter
↓
Hydra Connector
↓
Hydra Node
↓
TxValid / TxInvalid
↓
Execution Engine Update
↓
Snapshot Event
↓
State Anchoring
```

External strategy flow:

```
Trading Strategy
↓
Alpaca MCP Server
↓
Hydra SDK
↓
Hydra Head
```

This ensures deterministic state evolution.

---

# 11. Sample Application Architecture

Example application deployment:

```
Trading UI
     ↓
Hydra Wall Street SDK
     ↓
+----------------------------+
| Execution Engine           |
| Snapshot Manager           |
| Market Data Adapter        |
| Alpaca Integration Layer   |
+----------------------------+
     ↓
Hydra Node
     ↓
Cardano L1
```

External systems:

```
Alpaca MCP Server
Alpaca Market Data API
Alpaca Trading API
```

Optional integrations:

- market data providers
- analytics dashboards
- CI replay testing
- algorithmic trading systems